

OBJECT-ORIENTED DESIGN

UML Class Diagrams & Class Relationships

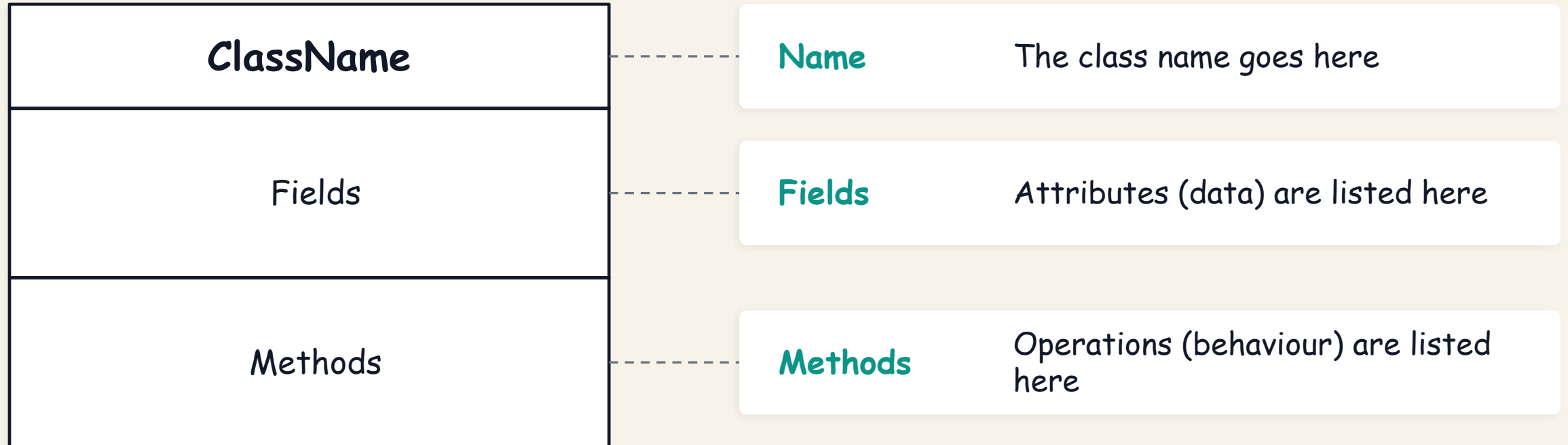
Class notation · UML → Java · Solid line = structure · Dashed = dependency / realization

Chapter Contents

- | | | |
|---|--------------------------|--|
| 1 | UML Class Diagram | Class box · Notation · UML → Java · Constructors |
| 2 | Overview | Purpose · Types of relationships |
| 3 | Association | Unidirectional · Bidirectional · Multiplicity |
| 4 | Aggregation | Whole-Part · UML · Example |
| 5 | Composition | Strong whole-part · Lifecycle · Example |
| 6 | Dependency | Uses · Parameter / local · Dashed UML |
| 7 | Generalization | Inheritance · Is-a · UML |
| 8 | Realization | Interface · Dashed UML |

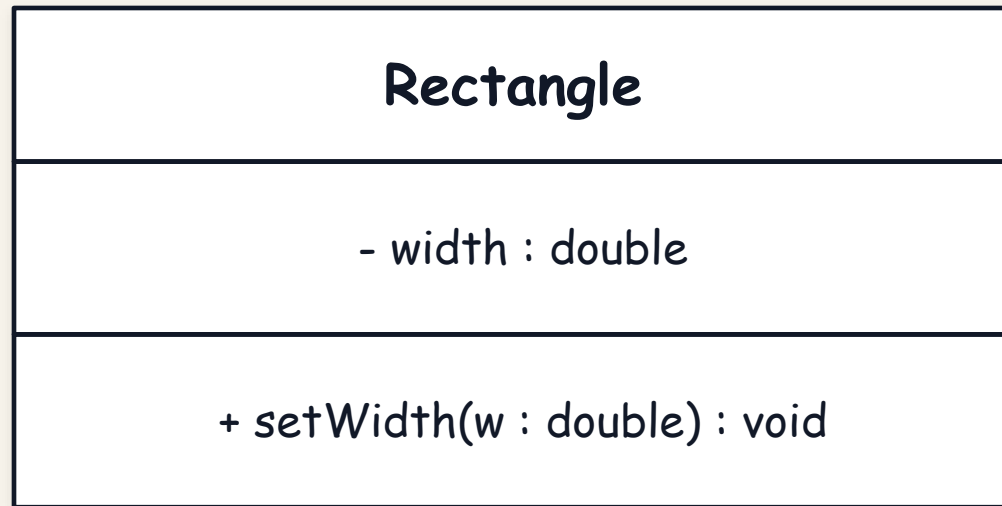
UML Diagram for a Class

Unified Modeling Language (UML) provides a set of standard diagrams for graphically depicting object-oriented systems.



UML Data Type and Parameter Notation

UML diagrams are language independent. They use their own notation to show data types, return types, access modifiers, etc.



Access modifiers

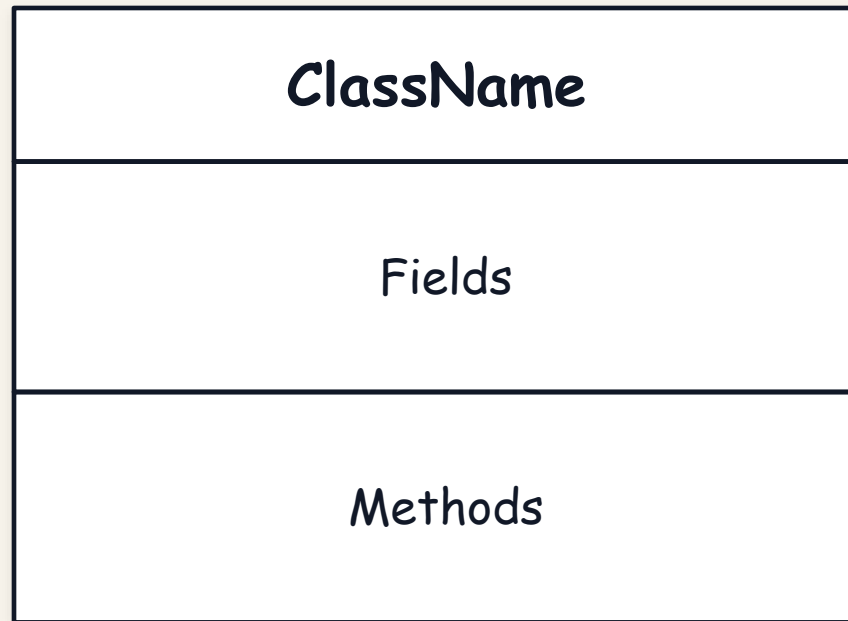
- + public
- private
- # protected

Field visibility name : type e.g. - width : double

Method visibility name(param : type) : returnType

Converting the UML Diagram to Code

Putting all of this information together, a Java class file can be built directly from the UML diagram — the parts match one to one.



```
class ClassName {  
    Fields  
    Methods  
}
```

Name → class header · Fields → member variables · Methods → method declarations

From UML to Java: the Rectangle Class

Rectangle
- width : double - length : double
+ setWidth(w : double) : void + setLength(len : double) : void + getWidth() : double + getLength() : double + getArea() : double

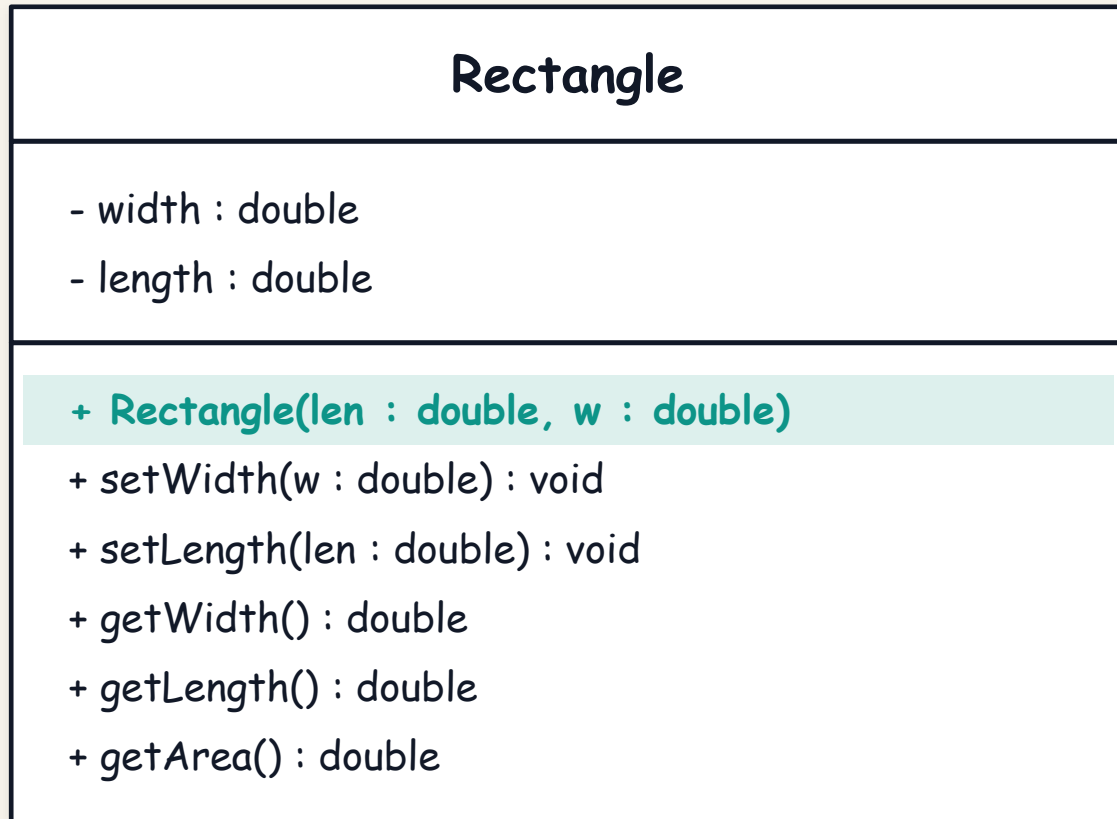
```
public class Rectangle {  
    private double width;  
    private double length;  
  
    public void setWidth(double w) { }  
    public void setLength(double len) { }  
    public double getWidth() { return 0.0; }  
    public double getLength() { return 0.0; }  
    public double getArea() { return 0.0; }  
}
```

Tip

The class skeleton can already be compiled and tested without method bodies — just return a dummy value (e.g. 0.0) from every non-void method.

Constructors in UML

In UML, a constructor is listed like a method, using the class name — but with no return type.



Note

Constructors have no return type — not even void.

```
public Rectangle(double len, double w) {  
    length = len;  
    width = w;  
}
```

Purpose of Class Relationships

Relationships describe collaboration, lasting structure, object lifetime, and change impact between classes.

Collaboration

Who works with whom?

Structure

Who holds a reference?

Lifetime

Do parts outlive the whole?

Change

Who is affected?

Types of Relationships

Type	Meaning	Line style
Association	A knows B	Solid
Aggregation	A has B (shared)	Solid + hollow ◇
Composition	A owns B	Solid + filled ◆
Dependency	A uses B briefly	Dashed arrow
Generalization	A is a B	Solid + △
Realization	A implements B	Dashed + △

Association

Structural “knows” link. UML uses a solid line. Direction (uni / bi) and multiplicity (1, 0..1, *, 1..*) refine the design.

UML

Solid line between class boxes

Code

Field of the other type

Direction

Arrow optional — shows navigability

Multiplicity

Numbers near each end of the line

Unidirectional Association



Many orders may refer to one customer · navigation Order → Customer only

```
public class Order {  
    private Customer customer; // multiplicity 1 on Customer side  
}  
  
// Customer has no List<Order> → unidirectional
```

Bidirectional Association



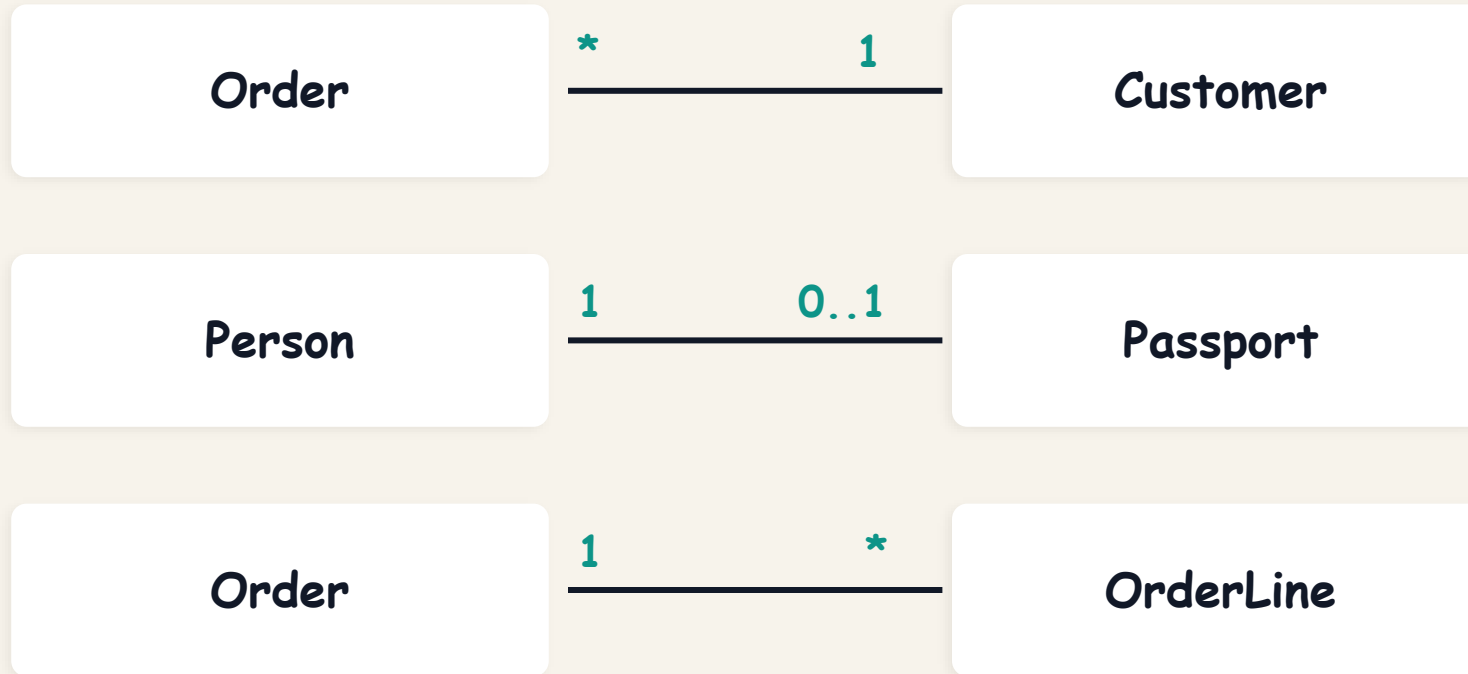
Many-to-many · both sides navigate · keep lists consistent

```
class Student { List<Course> courses; } // * courses
class Course { List<Student> students; } // * students
// enroll() must update BOTH sides
```

Multiplicity — Numbers on the Line

Notation	Meaning	Typical field
1	Exactly one	Customer customer;
0..1	Zero or one	Passport passport; // optional
*	Zero or more	List<OrderLine> lines;
1..*	One or more	List + enforce non-empty
n	Exactly n	Rare fixed size

Multiplicity on Diagrams



Place numbers near each end of the solid line — they belong to that side's count.

Aggregation — Whole-Part (shared)



Hollow diamond on the whole · solid line · parts may be shared

```
public class Department {  
    private List<Employee> staff = new ArrayList<>(); // *  
    public void add(Employee e) { staff.add(e); }  
}  
  
// Employee can move to another Department
```

Composition — Strong Whole-Part



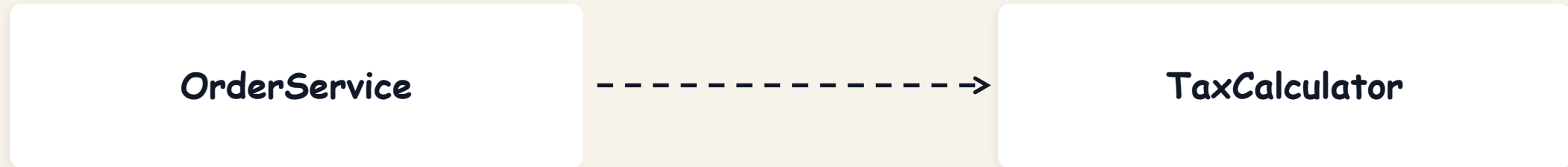
Filled diamond on the whole · exclusive ownership · bound lifetime

```
public class Order {  
    private final List<OrderLine> lines = new ArrayList<>();  
    public void addLine(String pid, int qty) {  
        lines.add(new OrderLine(pid, qty)); // created by whole  
    }  
}
```

Aggregation vs Composition

Aspect	Aggregation	Composition
Ownership	Shared	Exclusive
Lifetime	Part independent	Bound to whole
UML	Hollow ◇ + solid	Filled ◆ + solid
Multiplicity	Often 1 — *	Often 1 — 1..*
Example	Dept-Employee	Order-OrderLine

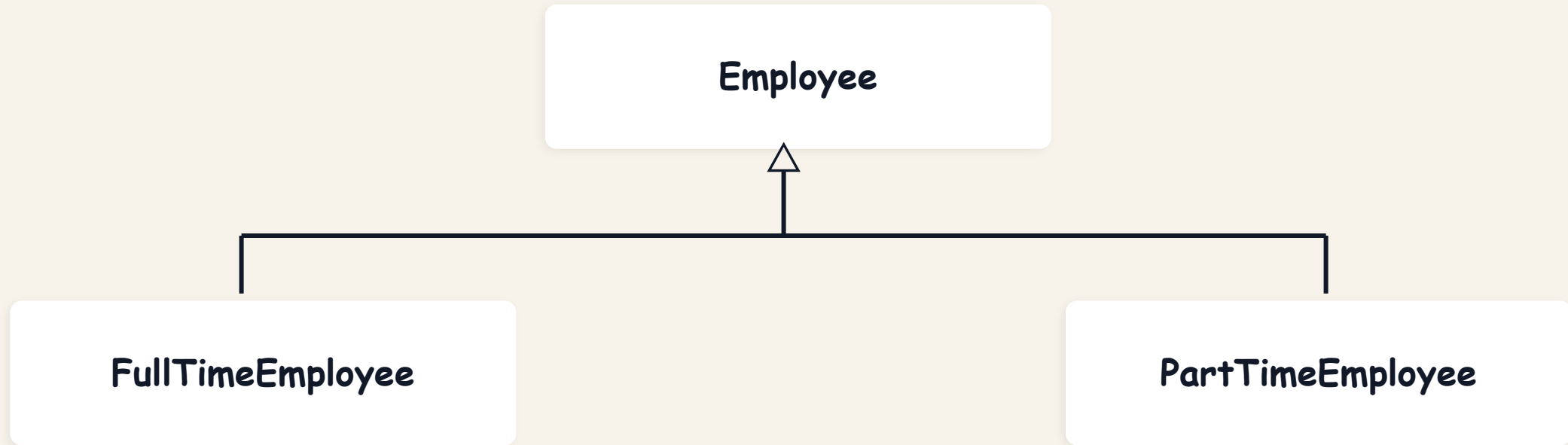
Dependency — Uses (dashed)



Dashed arrow toward the supplier · no lasting field

```
public double checkout(Order order, TaxCalculator tax) {  
    return tax.apply(order.subtotal()); // parameter = dependency  
}
```

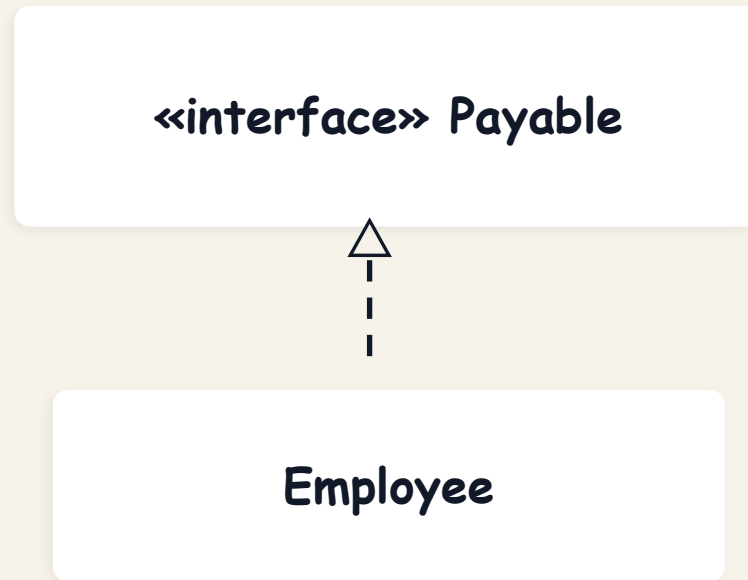
Generalization — Is-a (solid + Δ)



```
public class FullTimeEmployee extends Employee { ... }
```

```
Employee e = new FullTimeEmployee(...); // is-a
```

Realization — Implements (dashed + $\triangle$)



```
public interface Payable { double pay(); }  
public class Employee implements Payable { ... }
```

Getters / Setters are omitted for brevity.

